

Министерство образования Республики Беларусь
УО “Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №7
По дисциплине: «СПП»
Вариант 9

Выполнила:
студент 3 курса
группы ПО-12
Кравченко М.А.
Проверил:
Кулик А.Д.

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений.

Задание 1. Построение графических примитивов и надписей

Создать классы Point и Line. Объявить список из n объектов класса Point. Для объекта класса Line определить, какие из объектов Point лежат на одной стороне от прямой линии и какие на другой. Реализовать ввод данных для объекта Line и случайное задание данных для объекта Point.

Код программы:

Point_line_app.py

"""

Лабораторная работа: Построение графических примитивов и надписей

Вариант 9: классы Point и Line. Определение положения точек относительно прямой.

"""

```
import tkinter as tk
from tkinter import ttk, messagebox
import random
from datetime import datetime
from PIL import ImageGrab
```

class Point:

"""Класс точки на плоскости."""

```
    def __init__(self, x: float = 0, y: float = 0):
        self.x = x
        self.y = y
```

```
    def __repr__(self):
        return f"Point({self.x:.1f}, {self.y:.1f})"
```

```
    def get_coords(self):
        return self.x, self.y
```

class Line:

"""Класс прямой линии."""

```
    def __init__(self, a: float = 0, b: float = 0, c: float = 0):
        if a == 0 and b == 0:
            raise ValueError("a и b не могут быть нулевыми")
        self.a = a
        self.b = b
        self.c = c
```

@classmethod

```
    def from_two_points(cls, p1: Point, p2: Point):
        if p1.x == p2.x and p1.y == p2.y:
            raise ValueError("Точки не должны совпадать")
        a = p1.y - p2.y
        b = p2.x - p1.x
        c = p1.x * p2.y - p2.x * p1.y
        return cls(a, b, c)
```

```
    def get_value(self, point: Point) -> float:
        return self.a * point.x + self.b * point.y + self.c
```

```
    def get_side(self, point: Point) -> int:
        val = self.get_value(point)
        if val > 0:
            return 1
        if val < 0:
            return -1
        return 0
```

```

def get_equation_str(self) -> str:
    parts = []
    if self.a != 0:
        if self.a == 1:
            parts.append("x")
        elif self.a == -1:
            parts.append("-x")
        else:
            parts.append(f'{self.a:.1f}x')
    if self.b != 0:
        if self.b > 0:
            parts.append("+")
        if self.b == 1:
            parts.append("y")
        elif self.b == -1:
            parts.append("-y")
        else:
            parts.append(f'{self.b:.1f}y')
    if self.c != 0:
        sign = "+" if self.c > 0 else ""
        parts.append(f'{sign} {self.c:.1f}')
    if not parts:
        return "0 = 0"
    return " ".join(parts) + " = 0"

```

```

class DrawingCanvas(tk.Canvas):
    """Канва для рисования."""

```

```

    def __init__(self, parent, width=700, height=600):
        super().__init__(parent, width=width, height=height, bg="white")
        self.points = []
        self.line = None
        self.zoom = 1.0
        self.offset_x = 0
        self.offset_y = 0
        self.last_x = None
        self.last_y = None

        self.bind("<Button-1>", self._on_press)
        self.bind("<B1-Motion>", self._on_drag)
        self.bind("<MouseWheel>", self._on_zoom)

    def set_points(self, points):
        self.points = points
        self._redraw()

    def set_line(self, line):
        self.line = line
        self._redraw()

    def clear_points(self):
        self.points = []
        self._redraw()

    def reset_view(self):
        self.zoom = 1.0
        self.offset_x = 0
        self.offset_y = 0
        self._redraw()

    def zoom_in(self):
        self.zoom *= 1.2
        self._redraw()

    def zoom_out(self):
        self.zoom /= 1.2
        self._redraw()

    def _on_press(self, event):
        self.last_x = event.x
        self.last_y = event.y

    def _on_drag(self, event):
        if self.last_x is None:
            return
        dx = event.x - self.last_x

```

```

dy = event.y - self.last_y
self.offset_x += dx / self.zoom
self.offset_y -= dy / self.zoom
self.last_x = event.x
self.last_y = event.y
self._redraw()

def _on_zoom(self, event):
    delta = event.delta
    if delta > 0:
        self.zoom *= 1.1
    else:
        self.zoom /= 1.1
    self.zoom = max(0.1, min(10.0, self.zoom))
    self._redraw()

def _transform(self, x, y):
    w = max(self.wininfo_width(), 700)
    h = max(self.wininfo_height(), 600)
    sx = w / 2 + (x + self.offset_x) * self.zoom
    sy = h / 2 - (y + self.offset_y) * self.zoom
    return int(sx), int(sy)

def _inverse(self, sx, sy):
    w = max(self.wininfo_width(), 700)
    h = max(self.wininfo_height(), 600)
    x = (sx - w / 2) / self.zoom - self.offset_x
    y = (h / 2 - sy) / self.zoom - self.offset_y
    return x, y

def _redraw(self):
    self.delete("all")
    w, h = self.wininfo_width(), self.wininfo_height()
    if w < 10 or h < 10:
        self.after(100, self._redraw)
        return
    self._draw_grid()
    if self.line:
        self._draw_line()
    self._draw_points()
    self._draw_info()

def _draw_grid(self):
    w, h = self.wininfo_width(), self.wininfo_height()
    step = max(20, min(100, int(50 / self.zoom)))

    x = -self.offset_x - (w / self.zoom)
    while x < w / self.zoom - self.offset_x:
        sx, _ = self._transform(x, 0)
        if 0 <= sx <= w:
            self.create_line(sx, 0, sx, h, fill="#ddd", width=1)
        x += step

    y = -self.offset_y - (h / self.zoom)
    while y < h / self.zoom - self.offset_y:
        _, sy = self._transform(0, y)
        if 0 <= sy <= h:
            self.create_line(0, sy, w, sy, fill="#ddd", width=1)
        y += step

    zx, zy = self._transform(0, 0)
    if 0 <= zx <= w:
        self.create_line(zx, 0, zx, h, fill="black", width=2)
    if 0 <= zy <= h:
        self.create_line(0, zy, w, zy, fill="black", width=2)

    if 0 <= zx <= w:
        self.create_text(zx + 5, 15, text="Y", font=("Arial", 10))
    if 0 <= zy <= h:
        self.create_text(w - 15, zy - 5, text="X", font=("Arial", 10))

def _draw_line(self):
    """Рисование прямой."""
    w = self.wininfo_width()
    h = self.wininfo_height()
    pts = []

```

```

# Вертикальная прямая (b = 0)
if abs(self.line.b) < 1e-9:
    x_val = -self.line.c / self.line.a
    sx, _ = self._transform(x_val, 0)
    if 0 <= sx <= w:
        pts.append((sx, 0))
        pts.append((sx, h))
else:
    # Горизонтальная или наклонная прямая
    for ys in [0, h]:
        y = self._inverse(0, ys)[1]
        x_val = (-self.line.c - self.line.b * y) / self.line.a
        if abs(self.line.a) > 1e-9:
            sx, _ = self._transform(x_val, y)
            if 0 <= sx <= w:
                pts.append((sx, ys))

    for xs in [0, w]:
        x = self._inverse(xs, 0)[0]
        y_val = (-self.line.c - self.line.a * x) / self.line.b
        if abs(self.line.b) > 1e-9:
            _, sy = self._transform(x, y_val)
            if 0 <= sy <= h:
                pts.append((xs, sy))

if len(pts) >= 2:
    self.create_line(pts[0][0], pts[0][1], pts[1][0], pts[1][1], fill="#0066cc", width=3)
    self.create_text(
        10, 30, anchor="nw", text=f"Прямая: {self.line.get_equation_str()}", fill="#0066cc", font=("Arial", 10)
    )

def _draw_points(self):
    if not self.points:
        return

    if self.line is None:
        r = max(3, int(6 / self.zoom))
        for p in self.points:
            x, y = self._transform(p.x, p.y)
            self.create_oval(x - r, y - r, x + r, y + r, fill="red", outline="black")
        return

    s1, s2, ol = [], [], []
    for p in self.points:
        side = self.line.get_side(p)
        if side == 1:
            s1.append(p)
        elif side == -1:
            s2.append(p)
        else:
            ol.append(p)

    r = max(3, int(6 / self.zoom))

    for p in s1:
        x, y = self._transform(p.x, p.y)
        self.create_oval(x - r, y - r, x + r, y + r, fill="green", outline="black")
    for p in s2:
        x, y = self._transform(p.x, p.y)
        self.create_oval(x - r, y - r, x + r, y + r, fill="blue", outline="black")
    for p in ol:
        x, y = self._transform(p.x, p.y)
        self.create_oval(x - r, y - r, x + r, y + r, fill="gray", outline="black")

    self.create_text(10, 60, anchor="nw", text=f"Зеленые: {len(s1)}", fill="green", font=("Arial", 10))
    self.create_text(10, 80, anchor="nw", text=f"Синие: {len(s2)}", fill="blue", font=("Arial", 10))
    self.create_text(10, 100, anchor="nw", text=f"Серые: {len(ol)}", fill="gray", font=("Arial", 10))

def _draw_info(self):
    h = self.wininfo_height()
    self.create_text(
        10,
        h - 30,
        anchor="sw",
        text=f"Точек: {len(self.points)} | Масштаб: {self.zoom:.1f}x",
        font=("Arial", 9),
        fill="gray",
    )

```

```

    )
    self.create_text(
        10,
        h - 15,
        anchor="sw",
        text="Управление: колесико - масштаб, ЛКМ - панорамирование",
        font=("Arial", 9),
        fill="gray",
    )

class App:
    """Главное приложение."""

    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Точки и прямая")
        self.root.geometry("1100x700")

        self.points = []
        self.line = None
        self.animation = False
        self.anim_id = None

        self._create_widgets()
        self.root.protocol("WM_DELETE_WINDOW", self._on_close)

    def _create_widgets(self):
        main = ttk.Frame(self.root)
        main.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        left = ttk.Frame(main, width=300)
        left.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 10))
        left.pack_propagate(False)

        self.canvas = DrawingCanvas(main)
        self.canvas.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        self._create_controls(left)

    def _create_controls(self, parent):
        frame = ttk.LabelFrame(parent, text="Точки")
        frame.pack(fill=tk.X, padx=5, pady=5)

        row = 0
        ttk.Label(frame, text="Количество:").grid(row=row, column=0, padx=5)
        self.points_count = tk.IntVar(value=10)
        ttk.Spinbox(frame, from_=0, to=100, textvariable=self.points_count, width=8).grid(row=row, column=1, padx=5)
        row += 1

        ttk.Button(frame, text="Сгенерировать случайные точки", command=self._gen_points).grid(
            row=row, column=0, columnspan=2, pady=5
        )
        row += 1
        ttk.Button(frame, text="Очистить все точки", command=self._clear_points).grid(
            row=row, column=0, columnspan=2, pady=5
        )
        row += 1

        ttk.Separator(frame, orient="horizontal").grid(row=row, column=0, columnspan=2, pady=10)
        row += 1

        ttk.Label(frame, text="Добавить точку:").grid(row=row, column=0, columnspan=2)
        row += 1
        ttk.Label(frame, text="X:").grid(row=row, column=0)
        self.point_x = tk.DoubleVar(value=0)
        ttk.Entry(frame, textvariable=self.point_x, width=8).grid(row=row, column=1)
        row += 1
        ttk.Label(frame, text="Y:").grid(row=row, column=0)
        self.point_y = tk.DoubleVar(value=0)
        ttk.Entry(frame, textvariable=self.point_y, width=8).grid(row=row, column=1)
        row += 1
        ttk.Button(frame, text="Добавить", command=self._add_point).grid(row=row, column=0, columnspan=2, pady=5)

        frame = ttk.LabelFrame(parent, text="Прямая")
        frame.pack(fill=tk.X, padx=5, pady=5)

```

```

row = 0
tk.Label(frame, text="ax + by + c = 0").grid(row=row, column=0, columnspan=2)
row += 1

tk.Label(frame, text="a:").grid(row=row, column=0)
self.line_a = tk.DoubleVar(value=1)
tk.Entry(frame, textvariable=self.line_a, width=8).grid(row=row, column=1)
row += 1
tk.Label(frame, text="b:").grid(row=row, column=0)
self.line_b = tk.DoubleVar(value=0)
tk.Entry(frame, textvariable=self.line_b, width=8).grid(row=row, column=1)
row += 1
tk.Label(frame, text="c:").grid(row=row, column=0)
self.line_c = tk.DoubleVar(value=0)
tk.Entry(frame, textvariable=self.line_c, width=8).grid(row=row, column=1)
row += 1

tk.Button(frame, text="Установить прямую", command=self._set_line).grid(
    row=row, column=0, columnspan=2, pady=5
)
row += 1

tk.Separator(frame, orient="horizontal").grid(row=row, column=0, columnspan=2, pady=10)
row += 1

tk.Label(frame, text="По двум точкам:").grid(row=row, column=0, columnspan=2)
row += 1

tk.Label(frame, text="X1:").grid(row=row, column=0)
self.p1_x = tk.DoubleVar(value=0)
tk.Entry(frame, textvariable=self.p1_x, width=8).grid(row=row, column=1)
row += 1
tk.Label(frame, text="Y1:").grid(row=row, column=0)
self.p1_y = tk.DoubleVar(value=0)
tk.Entry(frame, textvariable=self.p1_y, width=8).grid(row=row, column=1)
row += 1
tk.Label(frame, text="X2:").grid(row=row, column=0)
self.p2_x = tk.DoubleVar(value=1)
tk.Entry(frame, textvariable=self.p2_x, width=8).grid(row=row, column=1)
row += 1
tk.Label(frame, text="Y2:").grid(row=row, column=0)
self.p2_y = tk.DoubleVar(value=1)
tk.Entry(frame, textvariable=self.p2_y, width=8).grid(row=row, column=1)
row += 1

tk.Button(frame, text="Установить по точкам", command=self._set_line_by_points).grid(
    row=row, column=0, columnspan=2, pady=5
)

frame = ttk.LabelFrame(parent, text="Вид")
frame.pack(fill=tk.X, padx=5, pady=5)

btn_frame = ttk.Frame(frame)
btn_frame.pack(pady=5)
tk.Button(btn_frame, text="+", width=3, command=self.canvas.zoom_in).pack(side=tk.LEFT, padx=2)
tk.Button(btn_frame, text="-", width=3, command=self.canvas.zoom_out).pack(side=tk.LEFT, padx=2)
tk.Button(btn_frame, text="Сброс", command=self.canvas.reset_view).pack(side=tk.LEFT, padx=2)

frame = ttk.LabelFrame(parent, text="Анимация")
frame.pack(fill=tk.X, padx=5, pady=5)

self.anim_btn = tk.Button(frame, text="Запустить", command=self._toggle_anim)
self.anim_btn.pack(pady=5)

sp_frame = ttk.Frame(frame)
sp_frame.pack(pady=5)
tk.Label(sp_frame, text="Скорость:").pack(side=tk.LEFT)
self.speed = tk.IntVar(value=50)
tk.Scale(sp_frame, from_=10, to=200, orient=tk.HORIZONTAL, variable=self.speed).pack(side=tk.LEFT, padx=5)

frame = ttk.LabelFrame(parent, text="Скриншоты")
frame.pack(fill=tk.X, padx=5, pady=5)
tk.Button(frame, text="Сделать скриншот", command=self._screenshot).pack(pady=5)

frame = ttk.LabelFrame(parent, text="Результаты")
frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

```

```

self.res_text = tk.Text(frame, height=12, wrap=tk.WORD)
self.res_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=5, pady=5)

sc = ttk.Scrollbar(frame, orient=tk.VERTICAL, command=self.res_text.yview)
sc.pack(side=tk.RIGHT, fill=tk.Y)
self.res_text.config(yscrollcommand=sc.set)

tk.Button(frame, text="Анализировать", command=self._analyze).pack(pady=5)

def _gen_points(self):
    cnt = self.points_count.get()
    self.points = []
    for _ in range(cnt):
        x = random.uniform(-15, 15)
        y = random.uniform(-15, 15)
        self.points.append(Point(x, y))
    self.canvas.set_points(self.points)
    self._update_results()

def _clear_points(self):
    self.points = []
    self.canvas.clear_points()
    self.res_text.delete(1.0, tk.END)

def _add_point(self):
    x = self.point_x.get()
    y = self.point_y.get()
    self.points.append(Point(x, y))
    self.canvas.set_points(self.points)
    self._update_results()

def _set_line(self):
    a = self.line_a.get()
    b = self.line_b.get()
    c = self.line_c.get()
    if a == 0 and b == 0:
        messagebox.showerror("Ошибка", "a и b не могут быть нулевыми")
        return
    self.line = Line(a, b, c)
    self.canvas.set_line(self.line)
    self._update_results()

def _set_line_by_points(self):
    x1, y1 = self.p1_x.get(), self.p1_y.get()
    x2, y2 = self.p2_x.get(), self.p2_y.get()
    if x1 == x2 and y1 == y2:
        messagebox.showerror("Ошибка", "Точки совпадают")
        return
    try:
        p1 = Point(x1, y1)
        p2 = Point(x2, y2)
        self.line = Line.from_two_points(p1, p2)
        self.canvas.set_line(self.line)
        self._update_results()
    except ValueError as e:
        messagebox.showerror("Ошибка", str(e))

def _update_results(self):
    if self.line and self.points:
        self._analyze()
    elif self.points:
        self.res_text.delete(1.0, tk.END)
        self.res_text.insert(tk.END, f"Всего точек: {len(self.points)}\n")
        self.res_text.insert(tk.END, "Прямая не задана")
    else:
        self.res_text.delete(1.0, tk.END)

def _analyze(self):
    if not self.line:
        messagebox.showwarning("Ошибка", "Задайте прямую")
        return
    if not self.points:
        messagebox.showwarning("Ошибка", "Нет точек")
        return

    s1, s2, o1 = [], [], []
    for p in self.points:

```



```

        side = self.line.get_side(p)
        if side == 1:
            s1.append(p)
        elif side == -1:
            s2.append(p)
        else:
            ol.append(p)

    self.res_text.delete(1.0, tk.END)
    self.res_text.insert(tk.END, "=" * 50 + "\n")
    self.res_text.insert(tk.END, "РЕЗУЛЬТАТЫ\n")
    self.res_text.insert(tk.END, "=" * 50 + "\n")
    self.res_text.insert(tk.END, f"Прямая: {self.line.get_equation_str()}\n\n")
    self.res_text.insert(tk.END, f"Сторона 1: {len(s1)}\n")
    self.res_text.insert(tk.END, f"Сторона 2: {len(s2)}\n")
    self.res_text.insert(tk.END, f"На прямой: {len(ol)}\n")

    if s1:
        self.res_text.insert(tk.END, "\nСторона 1:\n")
        for p in s1[:10]:
            self.res_text.insert(tk.END, f" {p}\n")
    if s2:
        self.res_text.insert(tk.END, "\nСторона 2:\n")
        for p in s2[:10]:
            self.res_text.insert(tk.END, f" {p}\n")
    if ol:
        self.res_text.insert(tk.END, "\nНа прямой:\n")
        for p in ol[:10]:
            self.res_text.insert(tk.END, f" {p}\n")

def _toggle_anim(self):
    if not self.points:
        messagebox.showwarning("Ошибка", "Нет точек")
        return
    if self.animation:
        self.animation = False
        self.anim_btn.config(text="Запустить")
        if self.anim_id:
            self.root.after_cancel(self.anim_id)
    else:
        self.animation = True
        self.anim_btn.config(text="Остановить")
        self._animate()

def _animate(self):
    if not self.animation:
        return
    for p in self.points:
        p.x += random.uniform(-0.3, 0.3)
        p.y += random.uniform(-0.3, 0.3)
    self.canvas.set_points(self.points)
    if self.line:
        self._analyze()
    self.anim_id = self.root.after(self.speed.get(), self._animate)

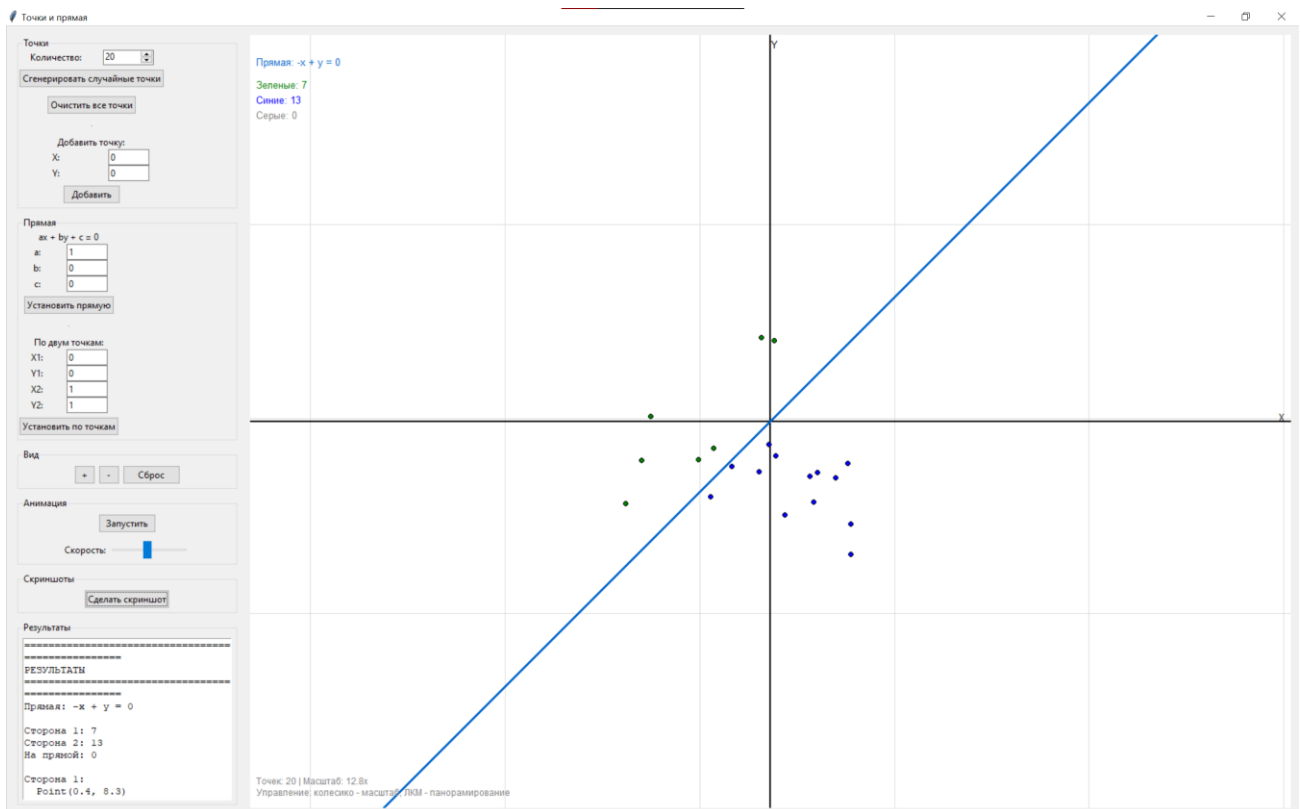
def _screenshot(self):
    fname = f"screenshot_{datetime.now().strftime('%Y%m%d_%H%M%S')}.png"
    x = self.canvas.winfo_rootx()
    y = self.canvas.winfo_rooty()
    w = self.canvas.winfo_width()
    h = self.canvas.winfo_height()
    img = ImageGrab.grab(bbox=(x, y, x + w, y + h))
    img.save(fname)
    messagebox.showinfo("Скриншот", f"Сохранен: {fname}")

def _on_close(self):
    self.animation = False
    self.root.destroy()

def run(self):
    self.root.mainloop()

if __name__ == "__main__":
    App().run()

```



Задание 2. Реализовать построение заданного типа фрактала по варианту Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

9) Остров Минковского

Код программы:

Island_minkowski.py

```
"""
Фрактал: Остров Минковского
"""

import tkinter as tk
from tkinter import ttk, messagebox
import math
from datetime import datetime
from PIL import ImageGrab

class MinkowskiIsland:
    """Генератор фрактала Острова Минковского."""

    def __init__(self, level: int, scale: float = 1.0):
        self.level = level
        self.scale = scale
        self.points = []

    def generate(self, cx: float, cy: float, size: float):
        """Генерация острова из 4 сторон."""
        self.points = []
        s = size
        self._gen_side(cx - s, cy - s, cx + s, cy - s)
        self._gen_side(cx + s, cy - s, cx + s, cy + s)
        self._gen_side(cx + s, cy + s, cx - s, cy + s)
        self._gen_side(cx - s, cy + s, cx - s, cy - s)
        return self.points
```

```

def _gen_side(self, x1: float, y1: float, x2: float, y2: float):
    """Рекурсивная генерация одной стороны."""
    self._gen_side_rec(x1, y1, x2, y2, self.level)

def _gen_side_rec(self, x1: float, y1: float, x2: float, y2: float, lvl: int):
    """Рекурсивная генерация с уровнем."""
    if lvl == 0:
        self.points.append((x1, y1))
        self.points.append((x2, y2))
        return

    dx = x2 - x1
    dy = y2 - y1
    length = math.hypot(dx, dy)

    if length < 0.1:
        self.points.append((x1, y1))
        self.points.append((x2, y2))
        return

    ux = dx / length
    uy = dy / length
    px = -uy
    py = ux
    spike = length / 3.0 * self.scale
    t = length / 3.0
    tt = length * 2.0 / 3.0

    p2x = x1 + ux * t
    p2y = y1 + uy * t
    p3x = p2x + px * spike
    p3y = p2y + py * spike
    p4x = p2x + ux * t + px * spike
    p4y = p2y + uy * t + py * spike
    p5x = x1 + ux * tt
    p5y = y1 + uy * tt
    p6x = p5x - px * spike
    p6y = p5y - py * spike
    p7x = p5x + ux * t - px * spike
    p7y = p5y + uy * t - py * spike

    nl = lvl - 1
    self._gen_side_rec(x1, y1, p2x, p2y, nl)
    self._gen_side_rec(p2x, p2y, p3x, p3y, nl)
    self._gen_side_rec(p3x, p3y, p4x, p4y, nl)
    self._gen_side_rec(p4x, p4y, p5x, p5y, nl)
    self._gen_side_rec(p5x, p5y, p6x, p6y, nl)
    self._gen_side_rec(p6x, p6y, p7x, p7y, nl)
    self._gen_side_rec(p7x, p7y, x2, y2, nl)

def get_point_count(self) -> int:
    """Вернуть количество точек."""
    return len(self.points)

def clear(self):
    """Очистить точки."""
    self.points = []

class FractalCanvas(tk.Canvas):
    """Канва для рисования фрактала."""

    def __init__(self, parent, width=800, height=600):
        super().__init__(parent, width=width, height=height, bg="white")
        self.points = []
        self.zoom = 1.0
        self.off_x = 0.0
        self.off_y = 0.0
        self.last_x = None
        self.last_y = None

        self.bind("<Button-1>", self._on_press)
        self.bind("<B1-Motion>", self._on_drag)
        self.bind("<MouseWheel>", self._on_zoom)

    def set_points(self, pts):

```

```

        """Установить точки для отрисовки."""
        self.points = pts
        self._redraw()

def reset(self):
    """Сбросить масштаб и позицию."""
    self.zoom = 1.0
    self.off_x = 0.0
    self.off_y = 0.0
    self._redraw()

def zoom_in(self):
    """Приблизить."""
    self.zoom *= 1.2
    self._redraw()

def zoom_out(self):
    """Отдалить."""
    self.zoom /= 1.2
    self._redraw()

def _on_press(self, e):
    self.last_x = e.x
    self.last_y = e.y

def _on_drag(self, e):
    if self.last_x is not None:
        dx = e.x - self.last_x
        dy = e.y - self.last_y
        self.off_x += dx / self.zoom
        self.off_y += dy / self.zoom
        self.last_x = e.x
        self.last_y = e.y
        self._redraw()

def _on_zoom(self, e):
    if e.delta > 0:
        self.zoom *= 1.1
    else:
        self.zoom /= 1.1
    self.zoom = max(0.1, min(10.0, self.zoom))
    self._redraw()

def _to_screen(self, x, y):
    w = self.winfo_width()
    h = self.winfo_height()
    sx = w // 2 + x * self.zoom + self.off_x * self.zoom
    sy = h // 2 + y * self.zoom + self.off_y * self.zoom
    return int(sx), int(sy)

def _redraw(self):
    self.delete("all")
    if len(self.points) < 2:
        return
    for i in range(len(self.points) - 1):
        x1, y1 = self._to_screen(self.points[i][0], self.points[i][1])
        x2, y2 = self._to_screen(self.points[i + 1][0], self.points[i + 1][1])
        self.create_line(x1, y1, x2, y2, fill="#0066cc", width=2)
    self._draw_info()

def _draw_info(self):
    h = self.winfo_height()
    txt = f"Точек: {len(self.points)} | Масштаб: {self.zoom:.1f}x"
    txt += " | Колесико - масштаб, ЛКМ - панорамирование"
    self.create_text(10, h - 20, anchor="sw", text=txt, font=("Arial", 9), fill="gray")

class App:
    """Главное окно."""

    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Фрактал - Остров Минковского")
        self.root.geometry("1200x700")

        self.level_var = tk.IntVar(value=2)
        self.size_var = tk.IntVar(value=200)

```

```

self.scale_var = tk.DoubleVar(value=1.0)

self.canvas = None
self._setup_ui()
self.root.protocol("WM_DELETE_WINDOW", self._on_close)

def _setup_ui(self):
    main = tk.Frame(self.root)
    main.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    left = ttk.Frame(main, width=300)
    left.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 10))
    left.pack_propagate(False)

    self.canvas = FractalCanvas(main, 800, 600)
    self.canvas.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

    self._make_controls(left)

def _make_controls(self, parent):
    # Параметры
    frame = ttk.LabelFrame(parent, text="Параметры")
    frame.pack(fill=tk.X, padx=5, pady=5)

    row = 0
    ttk.Label(frame, text="Глубина (0-5):").grid(row=row, column=0, padx=5, pady=5)
    ttk.Spinbox(frame, from_=0, to=5, textvariable=self.level_var, width=8).grid(row=row, column=1, padx=5, pady=5)
    row += 1

    ttk.Label(frame, text="Размер (50-400):").grid(row=row, column=0, padx=5, pady=5)
    ttk.Spinbox(frame, from_=50, to=400, textvariable=self.size_var, width=8).grid(
        row=row, column=1, padx=5, pady=5
    )
    row += 1

    ttk.Label(frame, text="Масштаб шипов:").grid(row=row, column=0, padx=5, pady=5)
    ttk.Spinbox(frame, from_=0.5, to=1.5, increment=0.1, textvariable=self.scale_var, width=8).grid(
        row=row, column=1, padx=5, pady=5
    )
    row += 1

    btn = ttk.Button(frame, text="Сгенерировать", command=self._gen)
    btn.grid(row=row, column=0, columnspan=2, pady=10, sticky="ew")

    # Вид
    frame = ttk.LabelFrame(parent, text="Вид")
    frame.pack(fill=tk.X, padx=5, pady=5)

    btn_frame = ttk.Frame(frame)
    btn_frame.pack(pady=5)
    btn_frame.pack(pady=5)
    ttk.Button(btn_frame, text="+", width=3, command=self.canvas.zoom_in).pack(side=tk.LEFT, padx=2)
    ttk.Button(btn_frame, text="-", width=3, command=self.canvas.zoom_out).pack(side=tk.LEFT, padx=2)
    ttk.Button(btn_frame, text="Сброс", width=5, command=self.canvas.reset).pack(side=tk.LEFT, padx=2)

    # Скриншот
    frame = ttk.LabelFrame(parent, text="Скриншот")
    frame.pack(fill=tk.X, padx=5, pady=5)
    ttk.Button(frame, text="Сделать скриншот", command=self._shot).pack(pady=5)

    # Инфо
    frame = ttk.LabelFrame(parent, text="О фрактале")
    frame.pack(fill=tk.X, padx=5, pady=5)

    txt = tk.Text(frame, height=8, wrap=tk.WORD)
    txt.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)
    txt.insert(
        tk.END,
        "Остров Минковского - классический фрактал.\n\n"
        "Глубина: количество итераций (0-5)\n\n"
        "Размер: сторона квадрата (50-400)\n\n"
        "Масштаб шипов: размер выступов (0.5-1.5)",
    )
    txt.config(state=tk.DISABLED)

def _gen(self):
    lvl = self.level_var.get()
    sz = self.size_var.get()

```

```

sc = self.scale_var.get()

if lvl < 0 or lvl > 5:
    messagebox.showerror("Ошибка", "Глубина от 0 до 5")
    return

fractal = MinkowskiIsland(lvl, sc)
pts = fractal.generate(0, 0, sz)
self.canvas.set_points(pts)
self.canvas.reset()

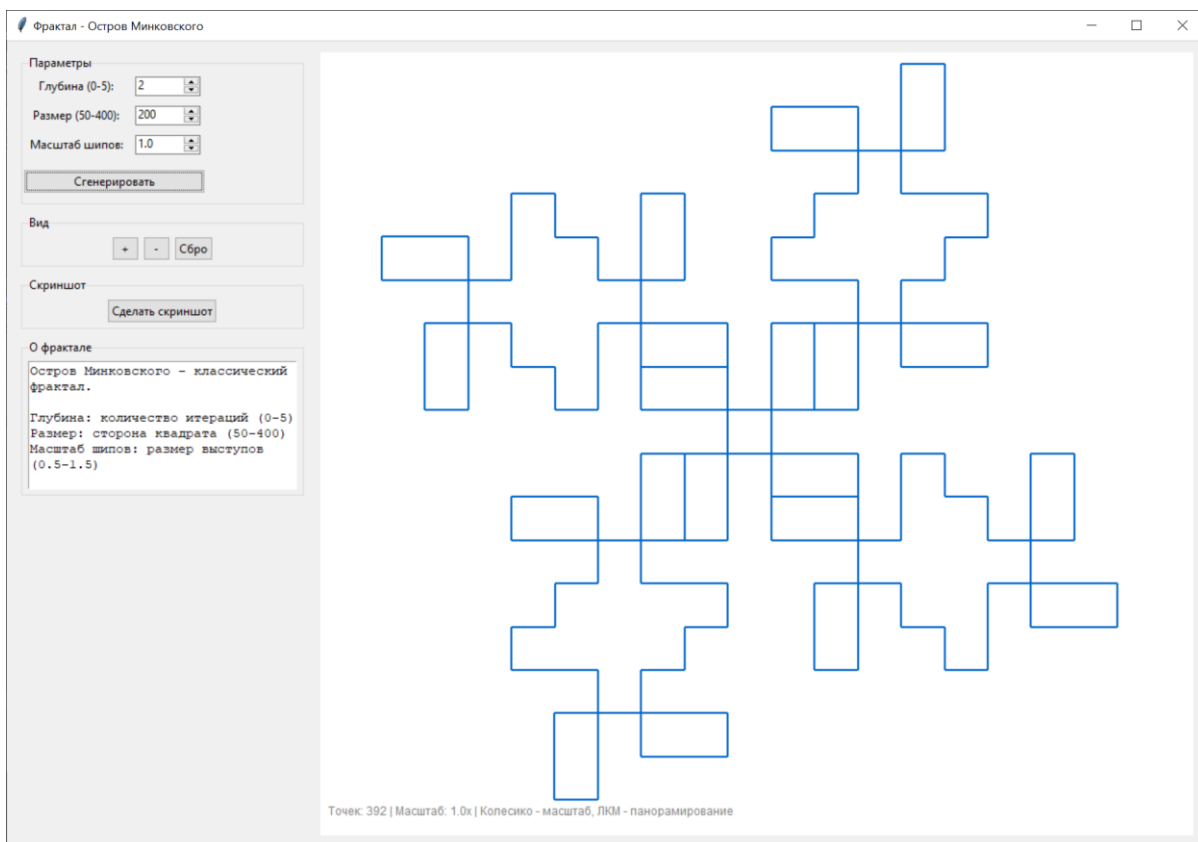
def _shot(self):
    name = f'fractal_{datetime.now().strftime("%Y%m%d_%H%M%S')}.png"
    x = self.canvas.winfo_rootx()
    y = self.canvas.winfo_rooty()
    w = self.canvas.winfo_width()
    h = self.canvas.winfo_height()
    img = ImageGrab.grab(bbox=(x, y, x + w, y + h))
    img.save(name)
    messagebox.showinfo("Скриншот", f'Сохранен: {name}')

def _on_close(self):
    self.root.destroy()

def run(self):
    """Запуск приложения."""
    self.root.mainloop()

if __name__ == "__main__":
    App().run()

```



Вывод: освоить возможности языка программирования Python в разработке оконных приложений.